

Our standalone test checks whether the FPGA can write to the USB3320 PHY and read a real response back. It uses a small RTL state machine, without Caliptra, MCU firmware, or the USB controller IP.

The path is:

PC Python → HTG940 J1 UART → FPGA FSM → daughter-card ULPI → FPGA → UART → PC

1. What Python sends

For example:

read 00

Python sends four UART bytes:

A5 01 00 00

Byte Meaning

A5 Start marker for our UART protocol

01 Read operation

00 PHY register address

00 Unused data byte for a read

This UART A5 marker is separate from writing the value A5 to the scratch register.

The FPGA receives the complete command in the reference-clock domain. It holds the address and data stable, then passes an operation request through a synchronizer to the ULPI-clock domain.

2. What the FSM does before sending

The FSM first waits for:

- **DIR=0:** the PHY is allowing the FPGA to own DATA.
- The previous DIR sample also being zero: the return turnaround cycle has finished.

The FPGA drives **DATA=00 while idle**. It then places the register command on DATA.

The direction control is:

drive = reset_n & ~dir & ~dir_q;

When physical **DIR becomes 1**, the FPGA immediately releases DATA. When DIR returns to zero, it waits through the turnaround cycle before driving again.

3. Register-read flow

For address 00, the FPGA sends **C0**:

C0 = 11000000

^^_____

read command + register address 00

The FSM sequence is:

State	FPGA action / condition
-------	-------------------------

WAIT_IDLE	Wait until the FPGA can own DATA
-----------	----------------------------------

CMD	Drive C0 and hold it until NXT is sampled high
-----	---

WAIT_DIR	Drive idle 00 , wait for the PHY to assert DIR
----------	---

READ	Release DATA, skip turnaround, capture DATA with DIR=1 and NXT=0
------	--

RETURN	Wait for DIR to return low and turnaround to finish
--------	---

DONE	Send the captured value and diagnostic information back to UART
------	---

For register 00, the expected captured value is **24**.

Reading C0 from the pins while the FPGA drives C0 would only be reading our own output. Successful capture requires PHY ownership of DATA.

4. Scratch-write flow

For:

write 16 A5

the FPGA performs:

1. Wait for the bus to be available.
2. Send **96**: immediate register-write command for address 16.
3. Wait for NXT to accept the command.

4. Drive the write data **A5**.
5. Wait for NXT to accept that data.
6. Drive **DATA=00** and **STP=1** for one ULPI cycle.
7. Return STP to zero.

Then read 16 sends **D6** and expects **A5** back.

A successful write handshake alone does not prove the register stored the value. **The following readback verifies that.**

5. Clocks and timing

Item	Value / purpose
FPGA reference clock	156.25 MHz , period 6.4 ns ; UART, startup delay, host timeout
PHY ULPI clock	60 MHz , period approximately 16.67 ns ; ULPI FSM and ILA
UART	115200 baud, 8N1
ULPI phase timeout	4096 cycles , approximately 68.3 μs

Overall FPGA reply timeout **100 ms**, so UART can report a missing/stopped ULPI clock

The FSM changes its registered outputs after a rising ULPI clock edge; the PHY samples them on subsequent rising edges.

The XDC also specifies external DATA/control input and output timing budgets. **Passing the functional test does not replace checking the final timing report**, especially since you changed timing failure to a warning.

6. Reset behavior

PB1 or PB2 resets our FPGA debug logic. After release, it waits approximately **10 ms** using the reference clock. The ULPI FSM then leaves reset through a three-stage synchronizer clocked by ULPI.

Our design does not send a physical reset to the daughter card. It has no BB22 reset output. According to your supplied daughter-card schematic, its supervisor generates the USB3320 RESETB signal.

7. Why we know it worked

Our test returned the correct ID bytes:

00 → 24

01 → 04

02 → 07

03 → 00

It also wrote and read back:

00, FF, A5, 5A, 01, 02, 04, 08, 10, 20, 40, 80

The matching scratch values show that the PHY received our writes and returned the stored data. The individual-bit patterns exercise every DATA bit in both directions.

This establishes working **ULPI register communication in this standalone design.**

```
PS C:\Users\furkanaydin\Documents\u2_uart_debug\u2_uart_debug> python ulpi_pc.py --port COM17 test
READ addr=0x00: OK; value=0x24; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
READ addr=0x01: OK; value=0x04; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
READ addr=0x02: OK; value=0x07; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
READ addr=0x03: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0xFF; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0xA5; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x5A; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x01; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x02; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x04; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
```

```
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0xA5; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x5A; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x01; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x02; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x04; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x08; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x10; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x20; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x40; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
READ addr=0x16: OK; value=0x80; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=1, read_captured=1, drive=1, last_bus=0x00
WRITE addr=0x16: OK; value=0x00; phase=idle; DIR=0; NXT=0; ULPI_recent_clock=1
  seen_NXT=1, seen_DIR=0, read_captured=0, drive=1, last_bus=0x00
PASS: USB3320 IDs and scratch read/write patterns
PS C:\Users\furkanaydin\Documents\u2_uart_debug\u2_uart_debug> |
```